class_definitions/paragraph.rb
```
paragraph = "any string can be a paragraph"

def paragraph.title?
  self.upcase == self
end

index(paragraph)
```

Now you can use plain strings in Bookworm and delete the `Paragraph` class.

Bill is awestruck by your solution. "I knew about Singleton Methods, but I never realized you could use them this way."

"Wait a minute," you reply. "You *knew* about them? What did you think they were useful for?"

"Singleton Methods aren't just useful for enhancing a specific object, like you just did." Bill replies. "They're also the basis for one of Ruby's most common features. What if I told you that you've been using Singleton Methods all along, without ever knowing it?"

Duck Typing

Some people are horrified by *Singleton Methods (114)*, thinking that if each object can have its own methods, no matter which class it belongs to, then your code is going to become a twisted tangle of spaghetti.

If you reacted that way yourself, then you're probably used to static languages. In a static language such as Java, you say that an object has type `T` because it belongs to class `T` (or because it implements interface `T`). In a dynamic language such as Ruby, the "type" of an object is not strictly related to its class. Instead, the "type" is simply the set of methods to which an object can respond.

People refer to this second, more fluid notion of a type as *duck typing*, referring to the saying: "if it walks like a duck and quacks like a duck, then it must be a duck." In other words, you don't care that an object is an instance of class `Duck`. You just care that it responds to `walk` and `quack`, whether they're regular methods, *Singleton Methods (114)*, or even *Ghost Methods (57)*.

If you hang around Ruby for a while, you will get used to duck typing—and after learning a few cool dynamic tricks, you might even wonder how you could have lived without it in the first place.

The Truth About Class Methods

Remember what you learned in *Inside the Object Model, on page 16*? Classes are just objects, and class names are just constants. If you remember this